

Arithmetic Expression Trees (Part 2)

Cpt S 321 Homework Assignment

Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Create a branch called "Branch_HW6" and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. Work on the Assignment Instructions below.
4. Create a pdf document with your AI statement for the HW (call it "AI_HW6"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.
5. When you are done, merge the branch back to the master.
6. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW6".
7. **Record a video to demo your HW (call it "Demo_HW6-FirstName_LastName").**
8. On Canvas -> Assignments -> Assignments -> **Upload your pre-recorded video by the HW deadline.**
9. Sign up for a live demo with the TAs before the deadline.

Assignment Instructions:

In this assignment you will finish what you started in the previous homework assignment by implementing an arithmetic expression parser that builds a tree for an expression. This tree can then be used for evaluation of the expression. You may want to refer back to the instructions for the first half of this assignment to make sure that everything was properly implemented there and that you are following the required naming conventions.

Recall that your expression tree class is implemented in your engine DLL and demoed in this assignment through a standalone console application that references the DLL. Integration into the spreadsheet will happen in a future homework assignment.

Requirement Details:

Support parentheses and operators with proper precedence:

- Support the addition and subtraction operators: + and -

- You may assume that all instances of the minus character are for subtraction. In other words, you don't have to support negation.
- Support the multiplication and division operators: * and /
- Each operator must be implemented with proper precedence.
- Parentheses must be supported and obeyed. Implementations without support for parentheses will result in a 50% deduction on all working operators.

Tree Construction:

- Build the expression tree correctly internally.
- You must use the Factory pattern seen in class.

Support for Variables:

- Support correct functionality of variables including multi-character values (like "A2").
- Variables will start with an alphabet character, upper or lower-case, and be followed by any number of alphabet characters and numerical digits (0-9).
- As you build the tree, every time you encounter a new variable add it to the dictionary of variable values with a default value of 0. This will be needed for integration with the spreadsheet application in a future assignment.
- Variables are stored per-expression, so creating a new expression should clear out the previous set of variable values. However, all variable values must persist as long as the expression is NOT being changed.
 - Ex: You shouldn't reset any variables after an evaluation or anything like that

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	● Fulfill all the requirements above with no inaccuracies in the output and no crashes.
CR2 Quality of Version Control (1 point)	● Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). ● Each commit should have cohesive functionality. ● Commit messages should concisely describe what is being committed. ● TDD should be used (i.e, write and commit tests first and then implement and commit functionality). ● Include "TDD" in all commit messages with tests that are written before the functionality is implemented. ● Use of a .gitignore. ● Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).

CR3: Quality of Identifiers (1 point)	• No underscores in names of classes, attributes, and properties. • No numbers in names of classes or tests. • Identifiers should be descriptive. • Project names should make sense. • Class names and method names use PascalCasing. • Method arguments and local variables use camelCasing. • No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	• Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. • All comment blocks use the format that is generated when typing “///” on the line above each entity. • There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	• Each file should only contain one public class. • Correct use of access modifiers. • Classes are cohesive. • Namespaces make sense. • Code is easy to follow. • StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. • Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	• Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. • Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). • <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	• Check the “Pre-recorded demos instructions” in the additional resources for details on the items below. • Version control history overview (< 30 sec.) • AI statement (< 30 sec.) • Design of the solution (2 min max) • Features showcase (2 min max)